



State Estimation

Observers and Output Feedback

Consultation: GP-S1111

Email: g.froessilva@qut.edu.au

Dr Guilherme Froes Silva
School of Electrical Engineering & Robotics
Queensland University of Technology

Overview

- The need for state estimation and observability.
- Discrete-Time Luenberger Observer.
- Observer Design via Pole Placement.
- Output Feedback Control: Combining Controller and Observer.
- The Separation and Certainty Equivalence principles.
- Observer-based control with reference tracking.
- Handling disturbances with observers (Integral Action revisited).
- Optimum Observer Design (Linear Quadratic Estimator – The Kalman Filter).

The Need for State Estimation

So far, we've designed controllers assuming **full state availability**:

- Pole Placement: $u(kT) = -Kx(kT)$.
- LQR: $u(kT) = -Kx(kT)$.
- Integral Action: $u(kT) = -K_x x(kT) - K_q q(kT)$.

State Availability Problem:

We often cannot measure all states directly and only have access to **outputs**:

$$y(kT) = Cx(kT) + Du(kT)$$

State Availability Problem

! Important

How can we implement state-feedback, $u(kT) = -Kx(kT)$, when $x(kT)$ is not directly measured?

Can we calculate the state from the output?

Given the output equation for a discrete-time system:

$$y(kT) = Cx(kT) + Du(kT)$$

Assuming $D = 0$ for simplicity, could we simply invert C ?

$$\hat{x}(kT) \stackrel{?}{=} C^{-1}y(kT)$$

Can we calculate the state from the output?

This is not possible in general!

As the matrix $C \in \mathbb{R}^{p \times n}$ is often **not square**.

Consider a system with two states ($n = 2$) and one output ($p = 1$):

$$\underbrace{\begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}}_{x(t) \in \mathbb{R}^n} = \begin{bmatrix} 0 & 1 \\ -0.1 & -0.2 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t)$$

$$\underbrace{y(t)}_{y(t) \in \mathbb{R}^p} = \begin{bmatrix} 1 & 0 \end{bmatrix}_{p \times n} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$$

Can we calculate the state from the input?

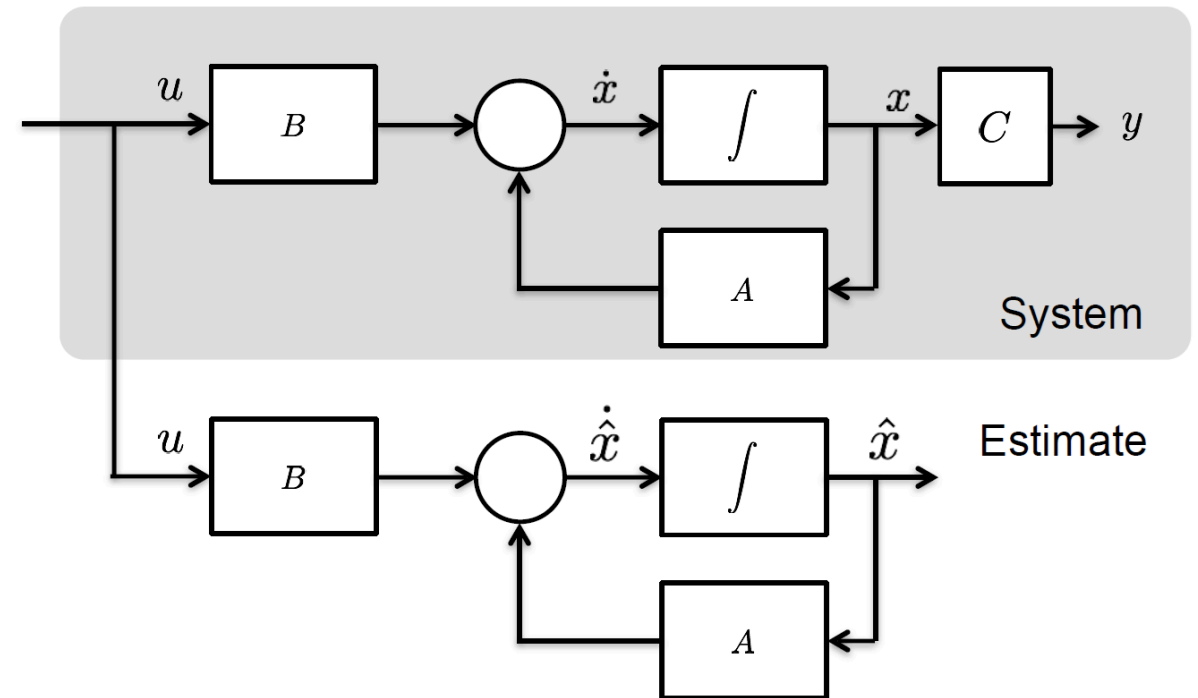
That is, can we simulate the system and use the states?

Consider the discrete-time system:

$$x(kT + T) = Gx(kT) + Hu(kT)$$

Let's build a "copy" of the system to estimate the state $\hat{x}(kT)$:

$$\hat{x}(kT + T) = G\hat{x}(kT) + Hu(kT)$$



This is an **open-loop observer** or **predictor**.

Open-loop estimation error

Define the estimation error $e(kT) = x(kT) - \hat{x}(kT)$.

Now, let's find the dynamics of the estimation error:

$$\begin{aligned} e(kT + T) &= x(kT + T) - \hat{x}(kT + T) \\ &= (Gx(kT) + Hu(kT)) - (G\hat{x}(kT) + Hu(kT)) \\ &= G(x(kT) - \hat{x}(kT)) \\ &= Ge(kT) \end{aligned}$$

Problem: The error dynamics are given by the open-loop system matrix G .

- If G has eigenvalues outside the unit circle (unstable system), the estimation error $e(kT)$ will **diverge**, regardless of the initial estimate $\hat{x}(0)$.
- Even if G is stable, the error might converge **too slowly** for effective control.

The Luenberger Observer



Idea

Use the **measured** output $y(kT)$ to *correct* the state estimate $\hat{x}(kT)$.

Discrete-Time Luenberger Observer

Consider the system:

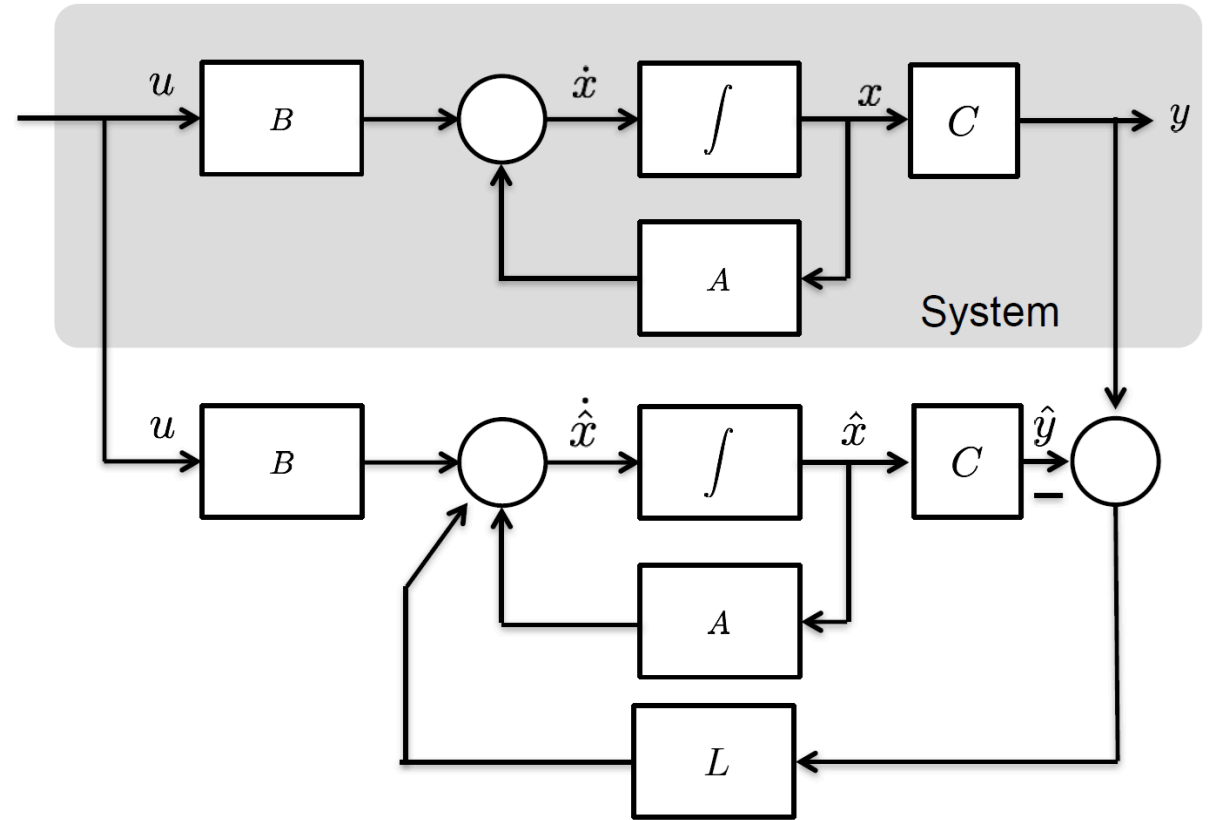
$$x(kT + T) = Gx(kT) + Hu(kT)$$

$$y(kT) = Cx(kT)$$

(Assuming $D = 0$ for simplicity, can be added back later).

Luenberger Observer:

$$\hat{x}(kT + T) = G\hat{x}(kT) + Hu(kT) + L(y(kT) - \hat{y}(kT))$$



- $\hat{x}(kT)$ is the state *estimate* at time kT .
- L is the **observer gain** matrix (to be designed).

- $y(kT)$ is the *actual* measured output.
- $\hat{y}(kT) = C\hat{x}(kT)$ is the *estimated* output.

Observer Error Dynamics:

The observer equation is:

$$\hat{x}(kT + T) = G\hat{x}(kT) + Hu(kT) + L(Cx(kT) - C\hat{x}(kT))$$

$$\hat{x}(kT + T) = (G - LC)\hat{x}(kT) + Hu(kT) + LCx(kT)$$

Now, let's find the dynamics of the estimation error $e(kT) = x(kT) - \hat{x}(kT)$:

$$\begin{aligned} e(kT + T) &= x(kT + T) - \hat{x}(kT + T) \\ &= (Gx(kT) + Hu(kT)) - ((G - LC)\hat{x}(kT) + Hu(kT) + LCx(kT)) \\ &= G(x(kT) - \hat{x}(kT)) - LC(x(kT) - \hat{x}(kT)) \\ &= (G - LC)e(kT) \end{aligned}$$

⚠ Important

The observer error dynamics are governed by the matrix $(G - LC)$. We want to choose the observer gain L such that the error $e(kT)$ converges to zero quickly, *i.e.*, the eigenvalues of $(G - LC)$ are stable (magnitude less than 1) and have desired dynamics.

Observability

! Question

Can we always find an observer gain L to place the eigenvalues of $(G - LC)$ arbitrarily?

Revisiting Observability

Analogy

For state feedback $u = -Kx$, we could arbitrarily place the eigenvalues of $(G - HK)$ *if and only if* the system (G, H) was controllable.

For observer design, the ability to place the eigenvalues of $(G - LC)$ depends on the system (G, C) being observable.

Observability

A discrete-time system, $x(kT + T) = Gx(kT) + Hu(kT)$, $y(kT) = Cx(kT)$, is completely observable if, for any initial time k_0T , the initial state $x(k_0T)$ can be *uniquely* determined from the knowledge of the input sequence $u(kT)$ and the output sequence $y(kT)$ for a finite number of steps.

Determining Observability

Theorem

The pair (G, C) is completely observable *if and only if* the observability matrix $\mathcal{O}$ has rank n , where n is the dimension of the state vector x .

How to check:

1. Construct the matrix $\mathcal{O} = [C \ CG \ CG^2 \ \dots \ CG^{n-1}]^T$.
2. Calculate its rank
 - `numpy.linalg.matrix_rank(ObsvMatrix)` (Python)
 - `rank(ObsvMatrix)` (MATLAB).
3. If $\mathcal{O}$ is square, it is full rank *if and only if* $\det(\mathcal{O}) \neq 0$.
4. If $\text{rank}(\mathcal{O}) < n$, the system is not completely observable.

Example: Observability Check

Consider a discrete-time model for a vehicle, with $m = 1, b = 0.5$.

Assume a sampling time $T = 0.1$.

Continuous:

$$A = \begin{bmatrix} 0 & 1 \\ 0 & -b/m \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & -0.5 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1/m \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Discretised (using `c2d(...)` in MATLAB):

$$G = \begin{bmatrix} 1 & 0 \\ 0.0975 & 0.9512 \end{bmatrix}, \quad H = \begin{bmatrix} 0.0049 \\ 0.0975 \end{bmatrix}.$$

Case 1: Position Measurement

Let $C_1 = \begin{bmatrix} 1 & 0 \end{bmatrix}$.

$$\mathcal{O}_1 = \begin{bmatrix} C_1 \\ C_1 G \end{bmatrix} = \begin{bmatrix} 1.0000 & 0 \\ 1.0000 & 0.0975 \end{bmatrix}.$$

$$\det(\mathcal{O}_1) = 1 \times 0.0975 - 0 \times 1 = 0.0975 \neq 0.$$

Rank is 2. System is observable when measuring position.

Case 2: Velocity Measurement

$C_2 = \begin{bmatrix} 0 & 1 \end{bmatrix}$.

$$\mathcal{O}_2 = \begin{bmatrix} C_2 \\ C_2 G \end{bmatrix} = \begin{bmatrix} 0 & 1.0000 \\ 0 & 0.9512 \end{bmatrix}.$$

$$\det(\mathcal{O}_2) = 0 \times 0.9512 - 0 \times 1 = 0.$$

Rank is 1. System is **not** observable when measuring velocity.

Observer Design via Pole Placement

Goal

Choose the observer gain L such that the error dynamics $e(kT + T) = (G - LC)e(kT)$ are stable and converge quickly.

Theorem

If the pair (G, C) is completely observable, then we can arbitrarily assign the eigenvalues of $(G - LC)$ by choosing the appropriate observer gain L .

Luenberger Observer: Design Steps

1. Check Observability: Verify that $\text{rank}(\mathcal{O}) = n$.
2. Choose Desired Observer Poles:
 - Select stable pole locations $z_1, z_2, \dots, z_n$ within the unit circle, $|z_i| < 1$.
 - These poles dictate the speed of convergence of the estimation error $e(kT)$.
 - Typically chosen 5-10 times faster (closer to the origin) than the desired controller poles.
3. Form Desired Characteristic Polynomial: $(z - z_1)(z - z_2) \cdots (z - z_n)$.
4. Determine Observer Gain L :
 - Find L such that the characteristic polynomial of $(G - LC)$ matches the desired characteristic polynomial. (`place` in MATLAB and `control.place` in Python).

Duality in Design

State-feedback control problem

Find K to place eigenvalues of $(G - HK)$.

This requires (G, H) to be controllable.

Observer design problem

Find L to place eigenvalues of $(G - LC)$.

This requires (G, C) to be observable.

Duality Property

The eigenvalues of a matrix are the same as the eigenvalues of its transpose. $\text{eig}(G - LC) = \text{eig}((G - LC)^\top) = \text{eig}(G^\top - C^\top L^\top)$

This means that designing a gain L to achieve desired poles for $(G - LC)$ is dual to designing a gain K_{obs} to achieve the same desired poles for $(G^\top - C^\top K_{obs}^\top)$.

Tip

Control Design:

`K = place(G, H, p)`

Observer Design:

`L = place(G', C', p)'`

Output Feedback Control

Idea

Now we combine the state-feedback controller with the state observer.

Since we cannot measure $x(kT)$, we use its estimate $\hat{x}(kT)$ (provided by the observer) in the control law.

- Observer: $\hat{x}(kT + T) = (G - LC)\hat{x}(kT) + Hu(kT) + Ly(kT)$
- Control Law: $u(kT) = -K\hat{x}(kT)$

K is the state-feedback gain (designed assuming $x(kT)$ was available)

L is the observer gain.

Closed-Loop Dynamics

The state variables are the actual system state $x(kT)$ and the estimation error $e(kT) = x(kT) - \hat{x}(kT)$.

$$x(kT + T) = Gx(kT) + Hu(kT)$$

$$u(kT) = -K\hat{x}(kT)$$

$$e(kT + T) = (G - LC)e(kT)$$

$$= -K(x(kT) - e(kT))$$

Substitute $u(kT)$ above:

$$x(kT + T) = Gx(kT) + H(-K(x(kT) - e(kT)))$$

$$= Gx(kT) - HKx(kT) + HKe(kT)$$

$$= (G - HK)x(kT) + HKe(kT)$$

Combining the dynamics for $x(kT)$ and $e(kT)$ in matrix form:

$$\begin{bmatrix} x(kT + T) \\ e(kT + T) \end{bmatrix} = \begin{bmatrix} G - HK & HK \\ 0 & G - LC \end{bmatrix} \begin{bmatrix} x(kT) \\ e(kT) \end{bmatrix}$$

Separation Principle

$$\begin{bmatrix} x(kT + T) \\ e(kT + T) \end{bmatrix} = \begin{bmatrix} G - HK & HK \\ 0 & G - LC \end{bmatrix} \begin{bmatrix} x(kT) \\ e(kT) \end{bmatrix}$$

The closed-loop system matrix is block triangular. Therefore, the eigenvalues of the overall closed-loop system are the union of:

- The eigenvalues of the state-feedback controller (eigenvalues of $G - HK$).
- The eigenvalues of the observer error dynamics (eigenvalues of $G - LC$).

Certainty Equivalence Principle

This means we can design the controller gain K and the observer gain L independently. The controller design determines the system response poles, and the observer design determines the error convergence poles.

Observer Design Considerations

- **Speed:** The state estimates should converge quickly so the controller can respond effectively.
- **Noise:** Large observer gains can amplify measurements noise, leading to poor performance.



Design Trade-off

Optimal observer design techniques (like Kalman filters) explicitly address this trade-off. For pole placement, choose poles that are fast enough but not excessively so.

Example: Observer-based Control

System: Vehicle from previous example.

$$G = \begin{bmatrix} 1.0000 & 0.0975 \\ 0 & 0.9512 \end{bmatrix}, \quad H = \begin{bmatrix} 0.0049 \\ 0.0975 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

1. Controller Design (Pole Placement), desired controller poles at $\{0.8, 0.7\}$:

$$K = \text{place}(G, H, [0.8, 0.7]) \rightarrow K = \begin{bmatrix} 6.1512 & 4.3159 \end{bmatrix}$$

2. Observer Design (Pole Placement), desired observer poles (faster than controller): $\{0.2, 0.3\}$.

$$L = \text{place}(G', C', [0.2, 0.3])' \rightarrow L = \begin{bmatrix} 1.4418 & 4.7007 \end{bmatrix}^T$$

3. Closed-Loop Simulation:

Implement the observer and controller $u(kT) = -K\hat{x}(kT)$. The overall system state includes x and e .

$$G_{cl} = \begin{bmatrix} G - HK & HK \\ 0 & G - LC \end{bmatrix}, \quad x(0) = [1, 0]^T.$$

Initial estimate error: $e(0) = x(0) - \hat{x}(0) = [1, 0]^T - [0, 0]^T = [1, 0]^T$ (assume observer starts at 0).

Extensions

- Observer-Based Control with Reference Tracking
- Integral Action with Observers
- Beyond Luenberger Observers: The Kalman Filter



Tip

You are not expected to know the mechanics/maths of the following techniques.

You **are expected** to know *what* they do, *why/when* they're useful, and *how* to use them (e.g. tuning).

You may consider these ideas to improve your design for the assessment.

Observer-Based Control with Reference Tracking

For non-zero setpoint tracking, $y(kT) \rightarrow r(kT)$, we can adapt the controller structure.

Using the *certainty equivalence*, we replace x with $\hat{x}$ in the setpoint regulation schemes discussed previously.

Using feedforward gain K_r :

$$u(kT) = -K\hat{x}(kT) + K_r r(kT)$$

Where K_r is chosen to ensure steady-state output matches $r(kT)$.

! Important

Finding K_r now requires considering the **combined controller-observer** dynamics. Often, integral action is preferred for robustness.

Integral Action with Observers

Observers can be used to estimate disturbances, which can then be used to effectively implement integral action.

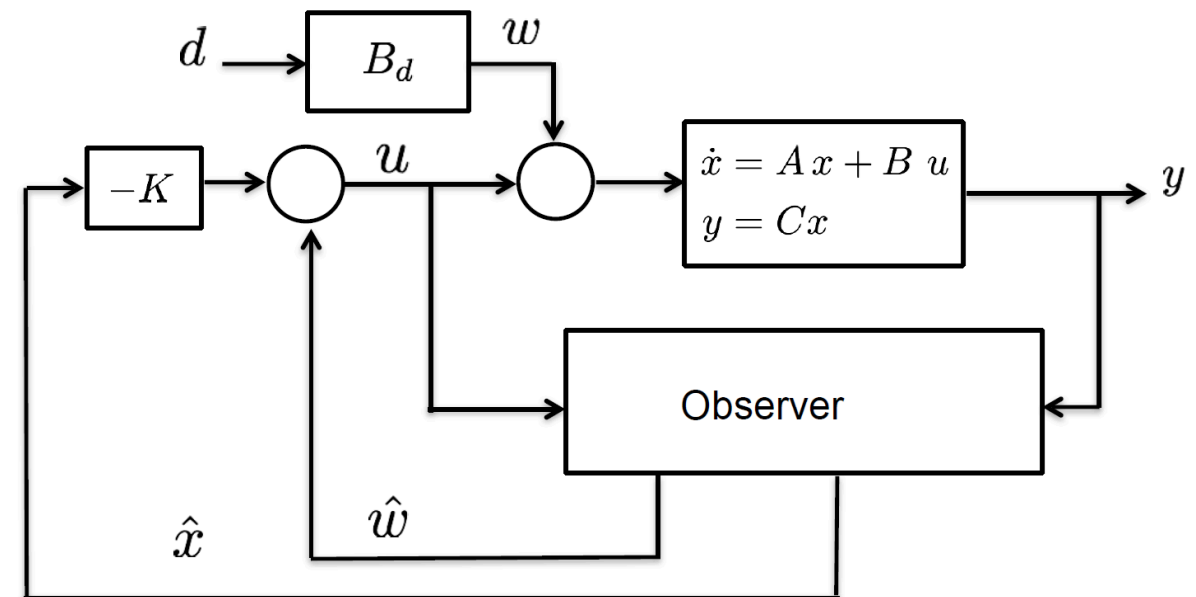
The idea is to augment the observer with a state that estimates the disturbance, $\hat{z}(kT) = [\hat{x}(kT), \hat{w}(kT)]^\top$.

$$\hat{z}(kT + T) = \begin{bmatrix} G & I \\ 0 & 1 \end{bmatrix} \hat{z}(kT) + \begin{bmatrix} H \\ 0 \end{bmatrix} u(kT),$$
$$\hat{y}(kT) = \begin{bmatrix} C & 0 \end{bmatrix} \hat{z}(kT)$$



Tip

Alternatively, you can simply use $\hat{x}(kT)$ and dynamic extension (like before) to achieve integral action.



Beyond Luenberger Observers

Limitations of Luenberger Observer:

- Assumes a **deterministic** system model (no noise).
- The observer gain L is chosen based on desired pole locations for the error dynamics, not based on noise characteristics.
- In the presence of significant **process noise** (uncertainties in the system dynamics) or **measurement noise** (sensor errors), the Luenberger observer's performance might not be optimal. Large gains (L) needed for fast convergence can amplify measurement noise.

Need for Stochastic Estimation:

- Real-world systems are inevitably affected by noise.
- We need methods that explicitly account for these uncertainties to achieve better state estimates.

The Kalman Filter (KF)

An **optimal estimator** for **linear systems** subject to **Gaussian noise**, which explicitly models:

- **Process Noise** $w(kT)$: Uncertainty in the system dynamics model. Characterised by covariance Q .
- **Measurement Noise** $v(kT)$: Uncertainty in the sensor measurements. Characterised by covariance R .

$$\begin{aligned}x(kT + T) &= Gx(kT) + Hu(kT) + w(kT) \\y(kT) &= Cx(kT) + v(kT)\end{aligned}$$

Key Ideas

- **Recursive:** Uses previous estimate and current measurement.
- **Predictor-Corrector Structure:**
 1. **Predict:** Project state estimate and error covariance forward in time based on the model (G, H, Q) .
 2. **Correct:** Update the predicted estimate using the actual measurement $y(kT)$, weighted by the **Kalman Gain** (K_k) .
- **Optimal Gain:** K_k is calculated at each step to minimise the estimated error covariance P_k , considering the relative uncertainties (Q vs R). It optimally balances trusting the model prediction vs. trusting the noisy measurement.

How the Kalman Filter Works: Predict & Correct

Prediction (Time Update):

Uses the system (G, H) and process noise covariance (Q) .

Goal: Predict $\hat{x}(kT)$ and error covariance $P(kT)$ at kT based on previous estimate.

Predict

$$\hat{x}^-(kT) = G\hat{x}(kT - T) + Hu(kT - T) \quad (\text{Predicted State})$$

$$P^-(kT) = GP(kT - T)G^T + Q \quad (\text{Predicted Covariance})$$

Update (Measurement Update / Correction):

Uses the measurement model (C) and measurement noise covariance (R) .

Goal: Correct the predicted state and error covariance using the actual measurement $y(kT)$.

Update

$$K_k = P^-(kT)C^T(CP^-(kT)C^T + R)^{-1} \quad (\text{Kalman Gain})$$

$$\hat{x}(kT) = \hat{x}^-(kT) + K_k(y(kT) - C\hat{x}^-(kT)) \quad (\text{Updated State})$$

$$P(kT) = (I - K_kC)P^-(kT) \quad (\text{Updated Covariance})$$

Cycle: The updated state $\hat{x}(kT)$ and covariance $P(kT)$ become the inputs for the prediction step at the next time instant $kT + T$.

KF finds the *optimal* gain based on noise statistics (Q, R) , while Luenberger uses pole placement.

Handling Nonlinearity: Extended Kalman Filter (EKF)

What if the system dynamics or measurement model are **nonlinear**?

$$\begin{aligned}x(kT + T) &= f(x(kT), u(kT)) + w(kT) \\ y(kT) &= h(x(kT)) + v(kT)\end{aligned}$$

Approach: Linearise the nonlinear functions f and h around the **state estimate** $\hat{x}(kT)$ at each time step.

$$G_k = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}(kT), u(kT)} \quad H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}(kT)}$$

Apply the standard KF equations using these time-varying Jacobians (G_k, H_k).



Tip

- EKF is **widely used** for nonlinear estimation (e.g., navigation, target tracking).
- EKF **does not guarantee optimality** like the linear KF. The linearization introduces approximations.
- EKF can **diverge** if nonlinearities are severe or initial estimates are poor.
- EKF is more **computationally intensive** due to Jacobian calculations at each step.
- Other nonlinear filters exist (e.g., Unscented Kalman Filter - UKF, Particle Filter) that avoid direct linearization but are computationally more complex. EKF is often a good starting point for mildly nonlinear systems.